

Application Modernization

Version 1, 2026-07-10

Software Factory and CI/CD

Introduction

Welcome to this quick course on the **Application Modernization**. This course is the **fourth** in a series of **six** courses about **Application Platform Delivery**:

- *Foundations: Developer Experience and GitOps Workflows*
- *Cloud-Native Development with Dev Spaces*
- *Software Factory and CI/CD*
- *Application Modernization(This course)*
- *Developer Hub and Platform Engineering*
- *Trusted Software Supply Chain (TSSC)*

What's in this course?

This training provides a comprehensive understanding of a modern Software Factory and its role in automating the software delivery lifecycle. Participants will learn how supporting services, Continuous Integration (CI), OpenShift Pipelines (Tekton), Continuous Delivery (CD), and promotion pipelines work together to build, validate, deploy, and promote applications across environments. By the end of the training, learners will understand how to implement standardized, automated, and GitOps-driven software delivery workflows on Red Hat OpenShift, enabling faster, more secure, and reliable application delivery.

Duration: 1.5 hours

Contributors

The PTL team acknowledges the valuable contributions of the following Red Hat associates:

- .. (SME)
- .. (SME)
- Yogita Soni (Content Architect)
- Anna Swicegood (Learning Product Manager)

Objectives

On completing this course, you should be able to:

Based on the section index, here are concise learning objectives:

Learning Objectives

By the end of this module, you will be able to:

- Integrate Dev Spaces with GitLab to enable source code management and developer workflows.
- Create and use Dev Spaces workspaces for cloud-native application development.
- Understand how Continuous Integration (CI) pipelines are triggered and executed from developer workspaces.
- Deploy applications to OpenShift using automated delivery workflows.
- Explain the role of promotion pipelines in advancing applications across development, testing, and production environments.

Prerequisites

This course assumes that you have the following prior experience:

1. [Red Hat OpenShift Container Platform \(OCP\) Technical Overview](#) or Kubernetes Basics

Lab Environment

Click the [RHDP](#) link above, log in to RHDP, and click **Order** to request a new [Service Enablement: App Platform Software Factory](#). On the order page for the catalog item, do the following:

- **Activity:** Select **Practice/Enablement**
- **Purpose:** Select **Learning about the Product**
- **Salesforce ID:** If you have an opportunity ID, enter it into this field, otherwise select **Project**, and then enter "Learning about RHCL" in the text field. You will see a warning message that a Salesforce ID is required, but you can safely ignore it.
- **Features:** Select **Enable Let's Encrypt**
- **Region:** Select the AWS region closest to your location
- **User count:** **1**
- **Control Plane Count:** **1**
- **OpenShift Version:** **4.16**
- **Control Plane Instance Type:** **m6a.4xlarge**
- Select the checkbox at the bottom of the page to confirm that lab costs will be charged to your cost center, and click **Order**.



The lab environment will take an hour to provision. You can check the status and details of the OpenShift cluster from the **Services** page of RHDP. The classroom is available for 48 hours initially. You can extend the duration of classroom availability by clicking **Help > Get Technical Help** in RHDP, and then requesting an extension by opening a ServiceNow ticket with the RHDP team.

RHDP Portal Links

- [RedHat Associates](#)
- [RedHat Partners](#)

Application Modernization

Why Modernize?

Many organizations have large portfolios of applications built on older frameworks and application servers. These applications work, but they carry increasing costs: slower release cycles, difficulty attracting developers, security patching burdens, and poor fit for container and Kubernetes-based deployment.

Application modernization is the process of updating these applications to use modern runtimes, frameworks, and deployment patterns — without rewriting from scratch.

Migration Toolkit for Applications (MTA)

[Red Hat Migration Toolkit for Applications \(MTA\)](#) accelerates large-scale application modernization by providing:

Analysis Engine

Scans application source code, configuration, and dependencies to identify migration issues. Produces detailed reports with effort estimates, mandatory changes, and optional improvements.

Rule Sets

Pre-built rules that detect patterns requiring changes when migrating between runtimes, frameworks, or deployment targets. Examples: JBoss EAP to Quarkus, Spring Boot version upgrades, Java EE to Jakarta EE, monolith to microservices, .NET Framework to .NET Core, etc.

Web Console

A UI for managing application portfolios, running assessments, and tracking migration progress across teams.

CLI (MTA CLI)

A command-line interface for running analysis as part of CI pipelines or developer workflows. Integrates with Dev Spaces for in-IDE analysis.

Modernization Strategies

MTA supports multiple modernization strategies depending on the application and business goals:

Rehost

Zero or near-zero code intervention. Purely infrastructure changes to package and deploy into the new target environment. Business logic and architecture remain unchanged. Fastest track.

Replatform

Move the application to containers and OpenShift with minimal code changes. Replace the legacy application server with a supported runtime like JBoss EAP on OpenShift. Balanced track.

Refactor

Deep architectural redesign to modern patterns like microservices or event-driven architectures. Adopt cloud-native patterns (health checks, externalized configuration, observability) and frameworks like Quarkus or Spring Boot. Most effort-intensive track.

Red Hat Runtimes

After modernization, applications run on a supported Red Hat Runtime:

- **Red Hat build of Quarkus** — Kubernetes-native Java framework optimized for fast startup and low memory
- **JBoss EAP** — Full Jakarta EE application server for enterprise workloads
- **Red Hat build of Spring Boot** — Supported distribution of Spring Boot with Red Hat libraries and security patches
- **Runtimes for Node.js, .NET, Go** — Supported runtimes for polyglot environments

How MTA Fits the Software Factory

In this training, MTA connects the modernization workflow to the CI/CD pipeline built in Lab 1:

1. **Analyze** — Run MTA against a legacy application to identify migration issues
2. **Modernize** — Apply code changes to adopt a Red Hat Runtime
3. **Integrate** — Push the modernized application into the existing CI pipeline
4. **Deploy** — Argo CD deploys the updated application through the GitOps workflow

Next Steps

Proceed to [Developer Hub Overview](#).

Application Modernization (MTA)

Overview

Use the Migration Toolkit for Applications (MTA) to analyze a legacy Java application, identify migration issues, and modernize it to run on a Red Hat Runtime. The modernized application will then be integrated into the CI pipeline built in Lab 1.

Install MTA

Install via OperatorHub

1. In the OpenShift Web Console, go to menu:Ecosystem[Software Catalog]
2. Search for **Migration Toolkit for Applications**
3. Click btn:[Install]

4. Accept the defaults and click btn:[Install]
5. Wait for the operator to reach the **Succeeded** phase

Verify from the CLI:

```
oc get csv -n openshift-mta | grep mta
```

Deploy the MTA Web Console

Create a Tackle CR to deploy the MTA web console:

```
oc apply -f - <<'EOF'
apiVersion: tackle.konveyor.io/v1alpha1
kind: Tackle
metadata:
  name: mta
  namespace: openshift-mta
spec: {}
EOF
```

Wait for the pods to be ready:

```
oc get pods -n openshift-mta -w
```

Get the MTA console route:

```
oc get route -n openshift-mta -o jsonpath='{range
.items[*]}https://{.spec.host}{"\n"}{end}'
```

Open the URL in your browser and login with the MTA credentials:

- Username: **admin**
- Password: **Passw0rd!**

Analyze the Application

Create an Application in MTA

1. In the MTA web console, go to menu:Applications[Application inventory]
2. Click btn:[Create new]
3. Fill in the application details:
 - **Name:** **parasol-legacy**
 - **Source repository:** Your GitLab repository URL for the legacy application

- **Branch:** `main`

4. Click btn:[Create]

Run the Analysis

1. Select the application and click btn:[Analyze]
2. Configure the analysis:
 - **Source:** Source code
 - **Target:** Quarkus
 - **Scope:** Application and internal dependencies
3. Click btn:[Run]

The analysis scans the source code, dependencies, and configuration files against MTA's rule sets. This may take a few minutes depending on the size of the application.

Review the Analysis Report

Once the analysis completes:

1. Click on the application name to view the report
2. Review the categories:
 - **Mandatory** — Changes required for the migration to succeed
 - **Optional** — Improvements that are recommended but not required
 - **Information** — Context and documentation links
3. Note the **Story Points** — an effort estimate for the migration

Common findings when migrating to Quarkus:

- Replace `javax. imports` with `jakarta.`
- Replace Spring-specific annotations with CDI equivalents
- Update persistence configuration from `persistence.xml` to `application.properties`
- Replace JAX-RS application class with Quarkus RESTEasy Reactive

CLI Analysis (Alternative)

You can also run analysis from the command line inside your Dev Spaces workspace:

```
mta-cli analyze \  
  --input /projects/parasol-legacy \  
  --output /tmp/mta-report \  
  --target quarkus
```

Open `/tmp/mta-report/index.html` in the IDE's preview to review results.

Modernize the Application

Apply the changes identified by MTA. Work through the mandatory issues first:

Update Dependencies

Update `pom.xml` to use Quarkus BOM and extensions:

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>io.quarkus.platform</groupId>
      <artifactId>quarkus-bom</artifactId>
      <version>${quarkus.platform.version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

Update Imports and Annotations

Replace `javax` with `jakarta` package imports:

```
// Before
import javax.inject.Inject;
import javax.ws.rs.GET;
import javax.ws.rs.Path;

// After
import jakarta.inject.Inject;
import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;
```

Update Configuration

Move configuration from XML files to `application.properties`:

```
quarkus.http.port=8080

# Disable Dev Services (use external services from Supporting Services lab)
%dev.quarkus.datasource.devservices.enabled=false
%dev.quarkus.kafka.devservices.enabled=false

# Development configuration (connects to external services in cluster)
%dev.quarkus.datasource.db-kind=postgresql
%dev.quarkus.datasource.username=${POSTGRES_USER:parasol}
%dev.quarkus.datasource.password=${POSTGRES_PASSWORD:parasol}
```

```
%dev.quarkus.datasource.jdbc.url=jdbc:postgresql://${POSTGRESQL_HOST:parasol-db.etx-
app-dev.svc.cluster.local}:5432/${POSTGRESQL_DATABASE:parasol}
%dev.kafka.bootstrap.servers=${KAFKA_BOOTSTRAP_SERVERS:parasol-kafka-kafka-
bootstrap.etx-app-dev.svc.cluster.local:9092}

# Production configuration
quarkus.datasource.db-kind=postgresql
%prod.quarkus.datasource.username=${POSTGRESQL_USER}
%prod.quarkus.datasource.password=${POSTGRESQL_PASSWORD}
%prod.quarkus.datasource.jdbc.url=jdbc:postgresql://${POSTGRESQL_HOST:parasol-
db}:5432/${POSTGRESQL_DATABASE:parasol}
%prod.kafka.bootstrap.servers=${KAFKA_BOOTSTRAP_SERVERS:parasol-kafka-kafka-
bootstrap:9092}

# Hibernate
quarkus.hibernate-orm.database.generation=update
```



Use the `%dev.` prefix for development-only settings and `%prod.` prefix for production-only settings. Dev Services are **disabled**—both development and production connect to external PostgreSQL and Kafka services deployed in **Deploy Supporting Services**. The DevSpaces devfile provides these environment variables automatically.



The PostgreSQL database deployed in the Supporting Services lab is shared across workspaces. The `update` mode shown above safely updates the schema without dropping existing tables. If you use `drop-and-create` instead, it will **delete and recreate the entire database schema on every application startup**, which may affect other workspaces or applications using the same database. Only use `drop-and-create` for isolated testing environments or deploy a separate database instance for applications that need destructive schema regeneration.

Validate the Build

Build and test the modernized application in your Dev Spaces workspace:

```
./mvnw clean package -DskipTests=false
```

Start the application in dev mode to verify it runs:

```
./mvnw compile quarkus:dev
```

Quarkus Dev Services will start a PostgreSQL container automatically.

Integrate with the CI Pipeline

Push the modernized application into the CI pipeline from Lab 1:

1. Commit and push the modernized source code to GitLab:

```
git add .
git commit -m "feat: modernize application to Quarkus"
git push
```

2. The GitLab webhook triggers the Tekton pipeline automatically
3. Monitor the pipeline run:

```
tkn pipelinerun logs -f -n <your_namespace>
```

4. Verify the pipeline completes all tasks:
 - Source clone
 - Maven build and test
 - Vault credential retrieval
 - Container image build and push
5. Confirm Argo CD syncs the updated image:

```
oc get application etx-app-dev -n etx-app-dev \
-o jsonpath='{.status.sync.status}'
```

6. Verify the modernized application is running:

```
oc get pods -n etx-app-dev
oc get route etx-app-base-app -n etx-app-dev -o jsonpath='{.spec.host}'
```

Open the route URL to confirm the modernized application is accessible and functional.

Summary

You have:

- Installed MTA and analyzed a legacy Java application
- Identified mandatory and optional migration issues with effort estimates
- Modernized the application to use Quarkus and Jakarta EE APIs
- Integrated the modernized application into the existing CI/CD pipeline
- Verified end-to-end deployment through the GitOps workflow

Next Steps

Proceed to [Install and configure Red Hat Developer Hub](#).